25/06/26
memory allocation algorithms for first fit,best fit, worst fit

code:

```
#include <stdio.h>

void firstFit(int block[], int m, int process[], int n) {
    int allocation[n];

    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {

            if(block[j] >= process[i]) {
                allocation[i] = j;
                block[j] -= process[i];
                break;
            }
        }
    }

    printf("\nFIRST FIT\n");
    for(int i = 0; i < n; i++) {
        printf("Process %d (%d KB) -> ", i+1, process[i]);

        if(allocation[i] != -1)
            printf("Block %d\n", allocation[i]+1);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int block[], int m, int process[], int n) {
    int allocation[n];

    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < n; i++) {

        int best = -1;

        for(int j = 0; j < m; j++) {

            if(block[j] >= process[i]) {

                if(best == -1 || block[j] < block[best])
                    best = j;
            }
```

```
        }

        if(best != -1) {
            allocation[i] = best;
            block[best] -= process[i];
        }
    }

    printf("\nBEST FIT\n");
    for(int i = 0; i < n; i++) {
        printf("Process %d (%d KB) -> ", i+1, process[i]);

        if(allocation[i] != -1)
            printf("Block %d\n", allocation[i]+1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int block[], int m, int process[], int n) {
    int allocation[n];

    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < n; i++) {

        int worst = -1;

        for(int j = 0; j < m; j++) {

            if(block[j] >= process[i]) {

                if(worst == -1 || block[j] > block[worst])
                    worst = j;
            }
        }

        if(worst != -1) {
            allocation[i] = worst;
            block[worst] -= process[i];
        }
    }

    printf("\nWORST FIT\n");
    for(int i = 0; i < n; i++) {
        printf("Process %d (%d KB) -> ", i+1, process[i]);

        if(allocation[i] != -1)
            printf("Block %d\n", allocation[i]+1);
        else
            printf("Not Allocated\n");
```

```
    }
}

int main() {

    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int block[m], b1[m], b2[m], b3[m];

    printf("Enter block sizes:\n");
    for(int i = 0; i < m; i++) {
        scanf("%d", &block[i]);

        b1[i] = block[i];
        b2[i] = block[i];
        b3[i] = block[i];
    }

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int process[n];

    printf("Enter process sizes:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &process[i]);

    firstFit(b1, m, process, n);
    bestFit(b2, m, process, n);
    worstFit(b3, m, process, n);

    return 0;
}
```
output:

```
Enter number of memory blocks: 5
Enter block sizes:
100 500 200 300 600
Enter number of processes: 4
Enter process sizes:
212 417 112 426

FIRST FIT
Process 1 (212 KB) -> Block 2
Process 2 (417 KB) -> Block 5
Process 3 (112 KB) -> Block 2
Process 4 (426 KB) -> Not Allocated

BEST FIT
Process 1 (212 KB) -> Block 4
Process 2 (417 KB) -> Block 2
Process 3 (112 KB) -> Block 3
Process 4 (426 KB) -> Block 5

WORST FIT
Process 1 (212 KB) -> Block 5
Process 2 (417 KB) -> Block 2
Process 3 (112 KB) -> Block 5
Process 4 (426 KB) -> Not Allocated

Process returned 0 (0x0)   execution time : 50.418 s
Press any key to continue.
```